

AI224: Operating Systems — Lecture 5

Dr. Karim Lounis

Jan - May 2025



Process Synchronization

Process Synchronization

Process Synchronization

What is synchronization? and Why do we care about it?



Process Synchronization

Motivation Example. Let X be a shared variable bwt Thread 1 and 2:

Thread 1

(A) $X:=1$;
 (B) ...
 (C) $\text{print}(X)$;

Thread 2

(D) $X:=2$;
 (E) ...
 (F) $X:=3$;

What is the value of X that is printed by Thread 1 when the following program is executed?

CoBegin

Thread 1();

Thread 2();

CoEnd

Process Synchronization

Motivation Example. Let X be a shared variable between Thread 1 and 2:

Thread 1

(A) $X:=1$;
(B) ...
(C) $\text{print}(X)$;

Thread 2

(D) $X:=2$;
(E) ...
(F) $X:=3$;

The value of X printed by Thread 1 can be: 1, 2, or 3. But which value is most likely?

Process Synchronization

Motivation Example. Let X be a shared variable between Thread 1 and 2:

Thread 1

(A) $X:=1$;
(B) ...
(C) $\text{print}(X)$;

Thread 2

(D) $X:=2$;
(E) ...
(F) $X:=3$;

The value of X printed by Thread 1 can be: 1, 2, or 3. But which value is most likely?

- Depends on the amount of code in (B) and (E).
- Depends on which thread started the execution first.
- Depends on the hardware, each thread is running on one CPU core.

Process Synchronization

Motivation Example. Let X be a shared variable between Thread 1 and 2:

Thread 1

(A) $X:=1$;
(B) ...
(C) $\text{print}(X)$;

Thread 2

(D) $X:=2$;
(E) ...
(F) $X:=3$;

The value of X printed by Thread 1 can be: 1, 2, or 3. But which value is most likely?

- Depends on the amount of code in (B) and (E).
- Depends on which thread started the execution first.
- Depends on the hardware, each thread is running on one CPU core.

The value of X is non-deterministic

Process Synchronization

Motivation Example. Let X be a shared variable between Thread 1 and 2:

Thread 1

(A) $X:=1$;
(B) ...
(C) $\text{print}(X)$;

Thread 2

(D) $X:=2$;
(E) ...
(F) $X:=3$;

The value of X printed by Thread 1 can be: 1, 2, or 3. But which value is most likely?

- Depends on the amount of code in (B) and (E).
- Depends on which thread started the execution first.
- Depends on the hardware, each thread is running on one CPU core.

The correct value of X depends on the logic of the program

Process Synchronization

Another Example. Let X be a shared variable between two Java threads, Thread 1 & Thread 2, where X is initially set to 5:

Thread 1

$X = X + 1;$

Thread 2

$X = X - 1;$

What is the final value of X when the following program is executed?

```
Thread Thread 1 = new Thread();
```

```
Thread Thread 2 = new Thread();
```

```
Thread 1.start();
```

```
Thread 2.start();
```

Process Synchronization

Another Example. Let X be a shared variable between two Java threads, Thread 1 & Thread 2, where X is initially set to 5:

Thread 1

$X = X + 1;$

Thread 2

$X = X - 1;$

The value of X could be: 4, 5, or 6. But which value is most likely?

Process Synchronization

Another Example. Let X be a shared variable between two Java threads, Thread 1 & Thread 2, where X is initially set to 5:

Thread 1

Thread 2

$X = X + 1;$

$X = X - 1;$

The value of i could be: 4, 5, or 6. But which value is most likely?

- Depends on which thread started first.
- Depends on which thread had the highest priority.
- Depends on the hardware, each thread is running on one CPU core.

Process Synchronization

Another Example. Let X be a shared variable between two Java threads, Thread 1 & Thread 2, where X is initially set to 5:

Thread 1

$X = X + 1;$

Thread 2

$X = X - 1;$

The value of X could be: 4, 5, or 6. But which value is most likely?

- Depends on which thread started first.
- Depends on which thread had the highest priority.
- Depends on the hardware, each thread is running on one CPU core.

The value of X is non-deterministic

Process Synchronization

Another Example. Let X be a shared variable between two Java threads, Thread 1 & Thread 2, where X is initially set to 5:

Thread 1

$X = X + 1;$

Thread 2

$X = X - 1;$

The value of X could be: 4, 5, or 6. But which value is most likely?

- Depends on which thread started first.
- Depends on which thread had the highest priority.
- Depends on the hardware, each thread is running on one CPU core.

The correct value of X depends on the logic of the program, which is known in this case. One thread increments X and the other one decrements it.

Process Synchronization

To Understand why the concurrent execution of two threads may result in three different values (4, 5, or 6), which should analyze their execution at the machine code level. For this we need to express their code in assembly.



Process Synchronization

Let us write the two statements:

$X = X + 1$; and $X = X - 1$;

in x86-assembly language, where AX is the accumulator and [0xB4FF55EF] is the address of the variable X in the central memory.

For the statement $X = X + 1$; it would be :

I_0 Mov AX, [0xB4FF55EF]

I_1 Inc AX

I_2 Mov [0xB4FF55EF], AX

For the statement $X = X - 1$; it would be :

I_3 Mov AX, [0xB4FF55EF]

I_4 Dec AX

I_5 Mov [0xB4FF55EF], AX

Process Synchronization

When the two threads are concurrently executed, their low-level instructions are interleaved in some order (set $X=n$):

I_0 Mov AX, [0xB4FF55EF]	$AX=n$
I_1 Inc AX	$AX=n+1$
I_3 Mov AX, [0xB4FF55EF]	$AX=n$
I_4 Dec AX	$AX=n-1$
I_2 Mov [0xB4FF55EF], AX	$X=n+1$
I_5 Mov [0xB4FF55EF], AX	$X=n-1$

The final state is $X = n-1$ (or $X = n+1$ if we switch I_2 & I_5). Both are incorrect states, since the correct value for the X should be n if the incrementer then the decrementer (or the reverse) are executed separately.

Problem of shared data corruption (compromising data coherency).

Process Synchronization

Problem of shared data corruption is due to:

- Threads can execute in (fake) concurrency, i.e., any process/thread can be interrupted at any point in its instruction flow (e.g., quantum expires, or I/O operation), and the CPU is assigned to another thread.
- Threads can execute in (real concurrency) parallel (e.g., two execution flows of two different processes/threads are simultaneously executing on separate CPUs or processing cores).
- These concurrent or parallel executing threads share some data.

The concurrent or parallel execution of multiple threads may result in the corruption of shared data by among several threads.

A multiprocessing system may reach an incorrect state if it allows the manipulation of shared data concurrently by multiple processes.

This computing phenomenon is known as **race condition**.

Process Synchronization

Definition

Race condition is when: (1) Multiple threads are being concurrently (i.e., in real or fake concurrency) executing (2) These threads share at least a common variable and (3) The outcome of the execution depends on the order in which those concurrent threads modified the shared variable.

How to deal with that:

We need a way to ensure that only one thread at a time can manipulate a shared variable among multiple concurrent threads.

Process Synchronization

Definition

Race condition is when: (1) Multiple threads are being concurrently (i.e., in real or fake concurrency) executing (2) These threads share at least a common variable and (3) The outcome of the execution depends on the order in which those concurrent threads modified the shared variable.

How to deal with that:

We need a way to ensure that only one thread at a time can manipulate a shared variable among multiple concurrent threads.

- Could we use a boolean variable?
- Maybe, we may use two boolean variables?
- Or maybe, we could use some special tools.

Process Synchronization

Use of a Boolean. Let *i* be a shared variable between two Java threads, Thread 1 & Thread 2, where *i* is initially set to 5. And Let *free* be a boolean initialized to true:

Thread 1

```
while(!free);
```

```
free=false;
```

```
i = i + 1;
```

```
free=true;
```

Thread 2

```
while(!free);
```

```
free=false;
```

```
i = i - 1;
```

```
free=true;
```

Process Synchronization

Use of a Boolean. Let i be a shared variable between two Java threads, Thread 1 & Thread 2, where i is initially set to 5. And Let $free$ be a boolean initialized to `true`:

Thread 1

```
while(!free);
```

```
free=false;
```

```
i = i + 1;
```

```
free=true;
```

Thread 2

```
while(!free);
```

```
free=false;
```

```
i = i - 1;
```

```
free=true;
```

This solution may work sometimes, but fails on certain execution scenarios.

Process Synchronization

Use of Two Booleans. Let i be a shared variable between two Java threads, Thread 1 & Thread 2, where i is initially set to 5. And Let `free1` and `free2` be two booleans initialized to `true`:

Thread 1

```
free1=false;
```

```
while(!free2);
```

```
i = i + 1;
```

```
free1=true;
```

Thread 2

```
free2=false;
```

```
while(!free1);
```

```
i = i - 1;
```

```
free2=true;
```

Process Synchronization

Use of Two Booleans. Let i be a shared variable between two Java threads, Thread 1 & Thread 2, where i is initially set to 5. And Let `free1` and `free2` be two booleans initialized to `true`:

Thread 1

```
free1=false;
```

```
while(!free2);
```

```
i = i + 1;
```

```
free1=true;
```

Thread 2

```
free2=false;
```

```
while(!free1);
```

```
i = i - 1;
```

```
free2=true;
```

This solution may work sometimes, but fails on certain execution scenarios and may cause a **deadlock**.

Process Synchronization

Definition

Process Synchronization. It is a multiprocessing paradigm that aims to managing and controlling the execution and the access to shared resources between multiple processes or threads.

- Important when the order of execution matters. I.e., when there is logic to enforce.
- Allows transparent process communication.
- Preserve data integrity (data coherency).
- Mechanisms such as: mutex, semaphores, monitors, Peterson's solution, and message passing can be used to enforce synchronization.

Critical section problem

Critical section (or region) problem

Critical Section

Definition

Critical Section: It is a part of a program code in which the program requests to use shared resources on which the access is mutually exclusive.

Any program that uses a shared resource — which access is mutually exclusive — is generally organized into three to five parts:

```
Program ( )
{
    [ Remainder Section ]
    Entry Section
        Critical Section
    Exit Section
    [ Remainder Section ]
}
```

In the [Entry Section](#), the process places a request to the OS to grant it access to the critical section, whereas in the [Exit Section](#), the process informs the OS of leaving the critical section to wake up any waiting processes.

Mutually-exclusive access to a resource (e.g., a variable) ensures that only one process or thread at a time may be granted write-mode access to that resource while any other process or thread requiring it be denied reading or writing access.

Critical Section

Definition

Critical Section: It is a part of a program code in which the program requests to use shared resources on which the access is mutually exclusive.

Example. Let **Thread 1** and **Thread 2** be two **concurrent** Java Threads:

Thread 1

`u = v + 50;`

`i = i + 15;`

`w = v % 20;`

`v = i / 10;`

Thread 2

`b = p - 51;`

`i = i - 15;`

`z = p * 90;`

`p = z % 10;`

Identify the critical sections for each thread.

Mutually-exclusive access to a resource (e.g., a variable) ensures that only one process or thread at a time may be granted write-mode access to that resource while any other process or thread requiring it be denied reading or writing access.

Critical Section

Definition

Critical Section: It is a part of a program code in which the program requests to use shared resources on which the access is mutually exclusive.

Example. Let **Thread 1** and **Thread 2** be two **concurrent** Java Threads:

Thread 1

`u = v + 50;`

`i = i + 15;`

`w = v % 20;`

`v = i / 10;`

Thread 2

`b = p - 51;`

`i = i - 15;`

`z = p * 90;`

`p = z % 10;`

Mutually-exclusive access to a resource (e.g., a variable) ensures that only one process or thread at a time may be granted write-mode access to that resource while any other process or thread requiring it be denied reading or writing access.

Critical Section

The Critical Section Problem: is all about designing a protocol that processes or threads can apply to cooperate and such protocol must satisfy the following three requirements:

- ✱ **Mutual Exclusion.** At most, one process or thread is executing its critical section at a time (i.e., **manipulating the shared exclusive-accessed resource**).
- ✱ **Progress.** A process or a thread that is outside its critical section (i.e., remainder section) should not block a process or thread from getting into their critical section.
- ✱ **Bounded Waiting.** A process or thread must not wait indefinitely in the queue (or indefinitely spinning around) to access its critical section.

-
- ☞ No assumptions should be made about speeds or number of CPUs.
 - ☞ A process may be interrupted during its critical section.
 - ☞ In “modern solutions”, there is a queue of PCBs (instead of spinning).

Synchronization Mechanisms

Synchronization Mechanisms

Mutex Locks

Definition

Mutex locks: Mutex for **Mutual exclusion**, is the simplest software-based mechanism (solution) for the critical section problem.

The mechanism consists of a **boolean variable** M (i.e., $M \in \{\text{true}, \text{false}\}$) that can be exclusively accessed and updated using two **primitives**.

✧ *It can be seen as: an open door to a room, that if closed and locked, no access to the room would be possible while the door is closed and locked. Access would be possible if the door is unlocked and opened again* ✧

These two primitives are: the `acquire()` ([Entry section code](#)) and the `release()` ([Exit section code](#)). — A.k.a., `lock()` | `unlock()`.

To access its **critical section**, a process must acquire the lock (i.e., execute `acquire(M)`). Also, that same process must release the lock (i.e., execute `release(M)`) when it is done executing the critical section (i.e., leaving it).

Mutex Locks

Thus, if the lock is available ($M=\text{true}$), then a process can acquire the lock and change its value (to $M=\text{false}$) and then enter the **critical section**.

If the lock is not available ($M=\text{false}$), then a process cannot acquire the lock and cannot enter the **critical section**. Instead, the process will actively keep on waiting for the lock to become available ($M=\text{true}$).

Below is a classical implementation of a mutex lock primitives:

```
acquire(M)
{
    while(! M);
    M = false;
}
```

```
release(M)
{
    M = true;
}
```

```
Program( )
{
    acquire(M);
    critical section;
    release(M);
}
```


Mutex Locks

Thus, if the lock is available ($M=\text{true}$), then a process can acquire the lock and change its value (to $M=\text{false}$) and then enter the **critical section**.

If the lock is not available ($M=\text{false}$), then a process cannot acquire the lock and cannot enter the **critical section**. Instead, the process will actively keep on waiting for the lock to become available ($M=\text{true}$).

Below is a classical implementation of a mutex lock primitives:

<pre> acquire(M) { while(! M); M = false; } </pre>	<pre> release(M) { M = true; } </pre>	<pre> Program() { acquire(M); critical section; release(M); } </pre>
--	---	--

Do we guarantee mutual exclusive access to CS with such a software implementation of mutex? — Initially, $M=\text{true}$.

Mutex Locks

Below is a classical implementation of a mutex lock primitives:

<pre>acquire(M) { while(! M); M = false; }</pre>	<pre>release(M) { M = true; }</pre>	<pre>Program() { acquire(M); critical section; release(M); }</pre>
--	---	---

Mutual exclusive access to CS may be violated:

- Process 1 runs the statement `while(!M);` and leaves the loop.
- Context-switching occurs, switching Process 1 with Process 2.
- Process 2 runs the statement `while(!M);` and leaves the loop.
- Process 2 updates the mutex lock `M=false`.
- Context-switching occurs, switching Process 2 with Process 1.
- Process 1 updates the mutex lock `M=false`.
- **Both Process 1 and 2 are now into the critical section 😊.**

Mutex Locks

```
acquire(M)
{
    while(! M);
    M = false;
}
```

```
release(M)
{
    M = true;
}
```

```
Program( )
{
    acquire(M);
    critical section;
    release(M);
}
```

Mutual exclusive access to CS may be violated:

- Process 1 runs the statement `while(!M);` and leaves the loop.
- Context-switching occurs, switching Process 1 with Process 2.
- Process 2 runs the statement `while(!M);` and leaves the loop.
- Process 2 updates the mutex lock `M=false`.
- Context-switching occurs, switching Process 2 with Process 1.
- Process 1 updates the mutex lock `M=false`.
- **Both Process 1 and 2 are now into the critical section** 😊.

The `acquire()` and `release()` primitives **must be atomic**.

Mutex Locks

If `M=false`, we would say the lock is held, acquired, unavailable, or locked. Else, we would say the lock is free, available, released, or unlocked.

The use of mutex locks creates **busy-waiting** (i.e., continuous looping or spinning). If a process finds the lock unavailable, it spins around (sometimes, the mutex lock is called **a spinlock** due to process spinning).

No context-switching is required when a process must wait on a lock. There is an active waiting that may waste CPU cycles.

Mutex is useful when the critical section is short (hence spinning time would be short and overhead would be good since no context-switching occurs).

A spinning process gets preempted after its allotted quantum expires, giving the chance to the lock-holder (process holding the lock) to release it.

Question

Could you think about some scenarios where the use of mutex may lead to a deadlock (Cue: bad coding || uniprocessor system)?

Mutex Locks (Atomic Implementation — Abstraction)

Test & Set (TAS) Instruction. The hardware would implement a CPU instruction that could write to a memory location and return its old value in an uninterrupted way (i.e., atomically — without being interrupted).

The atomic hardware instruction can be logically represented as:

<pre>int test_and_set(int * L) { int prev = *L; *L = 0; return prev; }</pre>	<pre>remainder section while (test_and_set(&lock) == 0); critical section lock = 1; //make it available remainder section</pre>
--	---

Here in the function, the previous content of the lock L gets loaded into one of the registers (e.g., AX) and then the memory content is set to 0 (L=0).

If two CPUs execute `test_and_set()` at the same time, the hardware will ensure that the two function calls happen atomically and sequentially.

Mutex Locks (Atomic Implementation)

Intel XCHG Instruction. On Intel architectures, the hardware provides the XCHG instruction to write to a memory location and return its old value in an uninterrupted way (E.g., `MOV AX, 0` then `XCHG AX, [0x0E3045]`).

The atomic hardware instruction can be logically represented as:

```
int XCHG(int * L, int v)
{
    int prev = *L;
    *L = v;
    return prev;
}
```

remainder section

```
while (XCHG(&lock, 0) == 0);
```

critical section

```
lock = 1; //make it available
```

remainder section

Here in the function, the previous content of the lock L gets loaded into one of the registers (e.g., AX) and then the memory content is set to 0 (L=0).

The execution of `XCHG()` on a CPU core is atomic and indivisible. However, the instruction does not lock access to the system bus.

Mutex Locks (Atomic Implementation)

Intel XCHG Instruction

Consider the following 8086 assembly code to be executed in a single-processor system where [0xF457] is the address of a mutex lock:

```
Mov Al, 0
```

```
Label: XCHG [0XF457], Al
```

```
Test Al, Al      //Test performs bitwise AND
```

```
Jz Label
```

```
... CS ...
```

```
Mov [0xF457], 1
```

It works fine because processor cannot switch from one process to another in the middle of XCHG instruction ([takes two indivisible CPU cycles](#)).

Mutex Locks (Atomic Implementation)

Intel XCHG Instruction

Consider the following 8086 assembly code to be executed in a single-processor system where [0xF457] is the address of a semaphore variable:

```
Mov Al, 0
```

```
Label: XCHG [0xF457], Al
```

```
Test Al, Al      //Test performs bitwise AND
```

```
Jz Label
```

```
... CS ...
```

```
Mov [0xF457], 1
```

But this may fail in a multiprocessor or multicore system

Mutex Locks (Atomic Implementation)

Intel XCHG Instruction

Consider the following 8086 assembly code to be executed in a single-processor system where [0xF457] is the address of a semaphore variable:

```
Mov Al, 0
```

```
Label: LOCK XCHG [0xF457], Al
```

```
Test Al, Al      //Test perform bitwise AND
```

```
Jz Label
```

```
... CS ...
```

```
Mov [0xF457], 1
```

Will work now on multiprocessor systems. By using LOCK the processor will have exclusive use of the bus till XCHG is completely executed. The LOCK prefix allows the CPU to lock access to the system bus.

Mutex Locks (Atomic Implementation)

Intel XCHG Instruction

Consider the following 8086 assembly code to be executed in a single-processor system where `[0xF457]` is the address of a semaphore variable:

RISC (Reduced Instruction Set Computing) processors (e.g., ARM, MIPS, RISC-V, etc) also have mechanisms to implement atomic operations:

- **LL/SC (Load-Linked.Store-Conditional).** Reads from memory location and monitor the memory location for possible changes. Then, write to the location only if no other thread has read or written to it.
- **Atomic Compare-and-Swap.** RISC-V provides the AMOSWAP (Atomic Memory Operation Swap) instruction, which atomically swaps a value in a register with a value in memory.

Will work now on multiprocessor systems. By using **LOCK** the processor will have exclusive use of the bus till **XCHG** is completely executed. The **LOCK** prefix allows the CPU to lock access to the system bus.

Mutex Locks (Back to Software-Level)

Example. Let i be a shared variable between two Java threads 1 & 2, where i is initially set to 5, and M is a mutex lock initialized to *true*:

Thread 1

```
acquire(M);
```

```
 $i = i + 1;$ 
```

```
release(M);
```

Thread 2

```
acquire(M);
```

```
 $i = i - 1;$ 
```

```
release(M);
```

What will be the value of i when the following program executes:

```
Thread Thread_1 = new Thread();
```

```
Thread Thread_2 = new Thread();
```

```
Thread_1.start();
```

```
Thread_2.start();
```

Mutex Locks (Back to Software-Level)

Example. Let i be a shared variable between two Java threads 1 & 2, where i is initially set to 5, and M is a mutex lock initialized to *true*:

Thread 1

```
acquire(M);
```

```
 $i = i + 1;$ 
```

```
release(M);
```

Thread 2

```
acquire(M);
```

```
 $i = i - 1;$ 
```

```
release(M);
```

What will be the value of i when the following program executes:

```
Thread Thread_1 = new Thread();
```

```
Thread Thread_2 = new Thread();
```

```
Thread_1.start();
```

```
Thread_2.start();
```

This time the value of i is deterministic and it is equal to 5.

Semaphores

Semaphores

Definition

Semaphore: Is a generalized software mechanism for the critical section problem and process synchronization — ordering (Edsger Dijkstra, 1962).

Consists of an **integer variable** S and two atomic primitives, $P()$ and $V()$, for its modification.

The primitive $P()$ (for Proberen — to test) allows to test the value of the semaphore and decrement it if greater or equal to 1, else wait.

The primitive $V()$ (for Verhogen— to increment) allows to increment the value of the semaphore.

The primitive $P()$ can also be denoted as $acquire()$, $wait()$, or $down()$. Similarly, the primitive $V()$ can be denoted as $release()$, $signal()$, or $up()$.



Semaphores

Definition

Semaphore: Is a generalized software mechanism for the critical section problem and process synchronization — ordering (Edsger Dijkstra, 1962).

Consists of an **integer variable** S and two atomic primitives, $P()$ and $V()$, for its modification. The primitive are classically implemented as follows:

$P(S)$	$V(S)$
{	{
while($S \leq 0$);	$S = S + 1$;
$S = S - 1$;	
}	}

The primitive $P()$ is used to acquire the semaphore M , whereas the primitive $V()$ is used to release the semaphore M .

Note the use of busy-waiting — the spinning instruction statement.

Semaphores

Definition

Semaphore: Is a generalized software mechanism for the critical section problem and process synchronization — ordering (Edsger Dijkstra, 1962).

Consists of an **integer variable** S and two atomic primitives, $P()$ and $V()$, for its modification. The primitive are classically implemented as follows:

$P(S)$ $\{$ while($S \leq 0$); $S = S - 1$; $\}$	$V(S)$ $\{$ $S = S + 1$; $\}$
---	---

The primitive $P()$ is used to acquire the semaphore M , whereas the primitive $V()$ is used to release the semaphore M .

What is the initial value of M ?

Semaphores

We distinguish two main types of semaphores:

- Binary semaphores are used when the semaphore value ranges only between 0 and 1 (i.e., Mutex lock — [particular case of semaphores](#)).
- Counting semaphores are used when the semaphore value ranges over an unrestricted domain (i.e., $M \in \mathbb{Z}$).

Also, we distinguish two styles of use:

- Mutual exclusion style, in which case M is initialized to $n \mid n \geq 1$.
- Waiting style, in which case M is initialized to 0.

Different implementations exist: The first one with busy waiting (spinning) and the second without busy waiting (through the use of a waiting-queue).

In the second implementation, a process can block itself with a system primitive `block()` when trying to acquire an unavailable semaphore and wake up another process (among the blocked processes) with a primitive `wake()` when releasing the semaphore.

Semaphores (Waiting style)

Example. Let Thread 1 and Thread 2 be two Java threads such that T1 should be executed after the termination of T2:

Thread 1

Code 1;

Thread 2

Code 2;

Such that:

```
Thread Thread_1 = new Thread();
```

```
Thread Thread_2 = new Thread();
```

```
Thread_1.start();
```

```
Thread_2.start();
```

The code is run on a multi-core (multiprocessor) computer system. Let us use a semaphore to enforce the process execution order.

Semaphores (Waiting style)

Example. Let T1 and T2 be two Java threads such that T1 should be executed after the termination of T2.

Thread 1

P(S);

Code 1;

Thread 2

Code 2;

V(S);

We use a semaphore S initialized to 0:

Semaphore S = 0;

Like this, Thread one will be stuck (spinning around) when trying to acquire the semaphore S. Thread 2 would run till completion, and then would release S. Next, Thread 1 would be able to acquire S and run.

Semaphores (Mutual exclusion style)

Example. Let i be a shared variable between two Java threads 1 & 2, where i is initially set to 5:

Thread 1

$P(S);$

$i = i + 1;$

$V(S);$

Thread 2

$P(S);$

$i = i - 1;$

$V(S);$

What would be the value of S so that $i=5$ when the following program executes and terminates?

```
Thread Thread_1 = new Thread();
```

```
Thread Thread_2 = new Thread();
```

```
Thread_1.start();
```

```
Thread_2.start();
```

Semaphores (Mutual exclusion style)

Example. Let i be a shared variable between two Java threads 1 & 2, where i is initially set to 5:

Thread 1

$P(S);$

$i = i + 1;$

$V(S);$

Thread 2

$P(S);$

$i = i - 1;$

$V(S);$

What would be the value of S so that $i=5$ when the following program executes and terminates?

```
Thread Thread_1 = new Thread();
```

```
Thread Thread_2 = new Thread();
```

```
Thread_1.start(); Thread_2.start();
```

The semaphore S must be **initialized to 1** (it is a mutex lock in this case). Then, $i=5$ at the end of the execution regardless of which thread ran first.

Semaphores (Mutual exclusion style)

Example (Counting Semaphore). Consider a petrol station with 6 petrol pumps, and a flow of vehicles is getting into the station to pump gas/diesel:

Vehicle_i

Begin

```
P(pump);
```

```
Pump_gas() || Pump_diesel();
```

```
V(pump);
```

End

What is the initial value of the semaphore pump?

Semaphores (Mutual exclusion style)

Example (Counting Semaphore). Consider a petrol station with 6 petrol pumps, and a flow of vehicles is getting into the station to pump gas/diesel:

Vehicle_i

Begin

```
P(pump);
```

```
Pump_gas() || Pump_diesel();
```

```
V(pump);
```

End

The semaphore `pump` should be initialized to 6.

Semaphores

Semaphore are powerful tools of process synchronization. However, so far, we have only seen semaphore implementations with busy-waiting.

To eliminate the busy waiting, semaphores are often implemented as follow:

- 1 An integer S that holds the number of instances of a given resource available for processes to use simultaneously.
- 2 A semaphore queue Q where blocked processes wait for the availability of a resource (critical section).

The semaphore can only be modified by two atomic primitives:

- 1 $P()$: tells the currently running process or thread to claim the control of a resource or make it wait if that resource is not available.
- 2 $V()$: Allows a process or thread to inform other processes or threads that it is done with that resource and to wake up a blocked process or thread in the queue if there are any.

Semaphores (Better Implementation)

The two primitives $P()$ and $V()$ become:

$P(S)$	$V(S)$
{	{
$S = S - 1;$	$S = S + 1;$
if($s < 0$)	if($S \leq 0$)
{	{
Block & place P in Q;	Wake up P from Q;
}	}
}	}

The primitive $P()$ decrements the value of S then: if $S < 0$ the “current” process is blocked and placed in the queue Q . Else it carries on.

The primitive $V()$ increments the value of S then: if $S \leq 0$ the “blocked” process in the head of the queue is dequeued and resumed.

Binary Semaphores (Enhanced mutex lock)

A binary semaphore is a semaphore which value can be 0 or 1.

- $P(S)$ nullifies the value of S and places all other processes to the waiting queue Q (semaphore queue of PCBs).
- $V(S)$ makes the critical section available by setting the value of S to 1 or releasing a “blocked” process from the queue Q .

$P(S)$

{

if($S==1$) $S = 0$;

Else

Block & place P in Q ;

}

$V(S)$

{

if(Q is $\emptyset$) $S = 1$;

Else

Wake up P from Q ;

}

- Each process, once dequeued and resumed will **straightforwardly** start executing its critical section.

Software solutions to the Critical Section Problem

- In 1962, the critical section problem was posed.
- In 1965, there was Dekker's Algo for two processes.
- In 1966, there was the first correct solution for n -processes.
- In 1968, THE OS by Dijkstra was the first OS that uses semaphores.
- In 1972, Eisenberg & McGuire's algo, a correct solution for n -processes.
- In 1974, Lamport's Bakery algo, a correct solution for n -processes.
- In 1981, Peterson's algo for two processes.

● End